FIAP ON

BUILDING STATISTICAL MODELS

PRÁTICAS DE REDES NEURAIS MULTICAMADAS

7

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Ciclo de vida de soluções baseadas em Ciência de Dados 6
Figura 2 – Saída da execução do código-fonte 2 10
Figura 3– Saída da execução do código-fonte 3 10
Figura 4 – Saída da execução do código-fonte 4 11
Figura 5 – Saída da execução do código-fonte 5 11
Figura 6 – Saída da execução do código-fonte 6 12
Figura 7 – Saída da execução do código-fonte 10 14
Figura 8 – Saída da execução do código-fonte 11 15
Figura 9 – Saída da execução do código-fonte 12 17
Figura 10 – Saída da execução do código-fonte 13 19
Figura 11 – Saída da execução do código-fonte 14 20
Figura 12 – Saída da execução do código-fonte 15 20
Figura 13 – Saída da execução do código-fonte 17 21
Figura 14 – Saída da execução do código-fonte 18 22
Figura 15 – Saída da execução do código-fonte 19 23
Figura 16 – Saída da execução do código-fonte 20 24
Figura 17 – Saída da execução do código-fonte 21 24
Figura 18 – Saída parcial (colunas suprimidas) da execução do código-fonte 22.....29
Figura 19 – Saída da execução do código-fonte 23 30
Figura 20 – Saída da execução do código-fonte 26 31
Figura 21– Saída da execução do código-fonte 27 32
Figura 22 – Saída da execução do código-fonte 28 34

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports necessários 9
Código-fonte 2– Leitura do dataset “insurance.csv” 9
Código-fonte 3 – Explorando o dataset “insurance.csv” 10
Código-fonte 4 – Contando os dados únicos da base “insurance.csv” 11
Código-fonte 5 – Remoção de duplicatas da base “insurance.csv” 11
Código-fonte 6 – Limpeza da base “insurance.csv” 12
Código-fonte 7– Separação de dados da base “insurance.csv” 13
Código-fonte 8– Gerando o modelo baseline (regressão linear) 13
Código-fonte 9 – Função de cálculo de métricas 14
Código-fonte 10 – Métricas para a regressão linear 14
Código-fonte 11 – Gerando o modelo RNA preliminar 15
Código-fonte 12 – Tunando o modelo RNA preliminar 16
Código-fonte 13 – Leitura do dataset “advertising.csv” 19
Código-fonte 14 – Explorando o dataset “advertising.csv” 19
Código-fonte 15 – Removendo dados duplicados do dataset “advertising.csv” 20
Código-fonte 16 – Separação de dados da base “advertising.csv” 21
Código-fonte 17 – Gerando o modelo baseline (regressão linear) 21
Código-fonte 18 – Gerando o modelo RNA preliminar 22
Código-fonte 19 – Gerando o modelo RNA secundário – Seed 97 23
Código-fonte 20 – Gerando o modelo RNA secundário – Seed 93 23
Código-fonte 21 – Gerando o modelo RNA secundário – Seed 3 24
Código-fonte 22 – Carregando do dataset “turnover.csv” 29
Código-fonte 23 – Plotando a distribuição das classes no dataset 29
Código-fonte 24 – Limpando os dados 30
Código-fonte 25 – Preparando os dados para aplicação de Machine Learning 31
Código-fonte 26 – Criando o modelo baseline 31
Código-fonte 27 – Criando o modelo baseline 32
Código-fonte 28 – Explicação do comportamento não linear do dataset 33

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

PRÁTICAS DE REDES NEURAIS MULTICAMADAS 5
1 INTRODUÇÃO 5
1.1 Motivação 5
1.2 Estrutura do capítulo 6
2 PROBLEMAS DE REGRESSÃO 7
2.1 Aplicação na Saúde 7
2.1.1 Definição do problema e dos dados 7
2.1.2 Primeiros passos 8
2.1.3 Primeiro modelo com Rede Neural Artificial 15
2.1.4 Aprimorando o modelo de Rede Neural Artificial 15
2.1.5 Considerações finais 17
2.2 Aplicação em Marketing 18
2.2.1 Definição do problema e dos dados 18
2.2.2 Primeiros passos 19
2.2.3 Primeiro modelo com Rede Neural Artificial 22
2.2.4 Aprimorando o modelo de Rede Neural Artificial 22
2.2.5 Considerações finais 25
3 PROBLEMAS DE CLASSIFICAÇÃO 26
3.1 Aplicação em Recursos Humanos 26
3.1.1 Definição do problema e dos dados 27
3.1.2 Primeiros passos 29
3.1.3 Primeiro modelo com Rede Neural Artificial 31
3.1.4 Aprimorando o modelo de Rede Neural Artificial 32
3.1.5 Considerações finais 34
4 CONCLUSÕES 35
5 HANDS ON: AVANÇANDO OS MODELOS CONSTRUÍDOS 36
5.1 Seu desafio 36
5.2 Proposta de solução 37
REFERÊNCIAS 38
GLOSSÁRIO 39

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

PRÁTICAS DE REDES NEURAIS MULTICAMADAS

1 INTRODUÇÃO

1.1 Motivação

Nesta aula, mergulharemos na prática da programação de redes neurais artificiais (RNAs), explorando o vasto potencial dessas poderosas ferramentas de modelagem.

A história das RNAs remonta ao Perceptron, que foi pioneiro no tratamento de problemas de classificação. Com o avanço das redes multicamadas, podemos abordar problemas mais sofisticados de classificação, os quais não são linearmente separáveis. Aprender os conceitos de RNAs com problemas de classificação é mais fácil, pois os problemas são mais bem compreendidos e a interpretação das métricas é mais simples e direta.

No entanto, **as RNAs também podem ser usadas para modelar problemas de regressão**, e é esse tipo de problema que daremos maior foco neste capítulo. Exploraremos casos de uso de RNAs em três áreas distintas: **saúde**, **marketing** e **recursos humanos**.

Referenciando o ciclo de vida das soluções de ciência de dados (Figura 1), cada case começará com uma contextualização da sua importância, seguindo pelas etapas típicas de leitura, limpeza e exploração de dados. A partir daí, faremos modelagens preditivas e as avaliaremos.

Considerando a natureza iterativa desse processo, sempre começaremos por um modelo inicial (baseline) baseado em aprendizado de máquina clássico, depois partiremos para um modelo preliminar de RNA e, posteriormente, refinaremos o modelo de RNA tornando-o mais robusto.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 1 – Ciclo de vida de soluções baseadas em Ciência de Dados
Fonte: https://medium.com/@vanshikagoel/data-science-life-cycle-in-5-steps-3fe720b2a2fe

1.2 Estrutura do capítulo

Este capítulo será inteiramente prático, permitindo que você mergulhe diretamente na codificação e experimente o poder das RNAs em suas próprias mãos. Ao estudar este capítulo, encorajamos você a programar ativamente e testar suas implementações no seu próprio ambiente de desenvolvimento. A prática constante é a chave para a maestria da programação de RNAs!

Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: Veja o material HOW TO para acessar o ambiente do Colab.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

DICA: Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

DICA: Utilizaremos 3 datasets distintos, então, fique atento qual arquivo de apoio foi citado para o carregamento dos dados!

2 PROBLEMAS DE REGRESSÃO

2.1 Aplicação na Saúde

Quando discutimos análises preditivas na saúde, é comum direcionarmos nosso foco para a identificação precoce de doenças e o aprimoramento dos cuidados clínicos. No entanto, é importante lembrar que a saúde abrange não apenas a assistência, mas também uma variedade de questões administrativas e operacionais.

Nesse contexto, a análise preditiva pode desempenhar um papel fundamental na identificação de padrões de utilização de serviços médicos, otimização de recursos, automatização de decisões e rotinas, entre vários outros exemplos.

2.1.1 Definição do problema e dos dados

Nosso problema é o desafio enfrentado pelos planos de saúde na estimativa de gastos por segurado. Poder estimar esse valor permite uma melhor alocação de recursos e uma gestão mais eficiente dos custos operacionais.

Do ponto de vista prático, utilizaremos uma base bastante conhecida contendo informações sobre os segurados (como idade, gênero, região de residência, entre outros). Esses atributos são comumente usados para prever os custos médicos de um

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

segurado, o que pode ser útil para companhias de seguros na precificação de apólices ou na identificação de segmentos de clientes com diferentes perfis de risco.

DICA: Para essa parte do código, utilizaremos o arquivo “insurance.csv”!

Essa base é composta pelos seguintes atributos:

- **age**: refere-se à idade do segurado, representada em anos;
- **sex**: indica o sexo do segurado, podendo ser 'male' ou 'female';
- **bmi**: corresponde ao Índice de Massa Corporal (IMC) do segurado, uma medida que relaciona o peso e a altura da pessoa, geralmente expressa em kg/m²;
- **children**: representa o número de filhos ou dependentes do segurado;
- **smoker**: indica se o segurado é fumante ou não, podendo ser 'yes' ou 'no';
- **region**: refere-se à região de residência do segurado, podendo ser 'northeast', 'northwest', 'southeast' ou 'southwest';
- **charges**: corresponde aos custos médicos do segurado, em dólares.

2.1.2 Primeiros passos

Referenciando o ciclo de vida das soluções de ciência de dados (Figura 1), já passamos pela etapa de entendimento do problema e obtenção dos dados. Agora, já aplicando programação, faremos a *leitura*, *exploração* e *preparação dos dados*, bem como criaremos o *modelo preditivo baseline*. Recomendamos que você programe junto conosco!

Nosso quadro de “imports” está grande, pois centralizamos nessa célula todos os imports necessários do capítulo. Ao executar essa célula, não será gerada saída.

```
# Ignorar warnings
import warnings
warnings.filterwarnings("ignore")

# Bibliotecas para uso e visualização de dados
import numpy as np
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Imports para seleção de modelos
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

# Imports para preparação de dados
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

# Imports para auxiliar a visualização de dados descritos em várias
variáveis
from sklearn.decomposition import PCA

# Import de modelos preditivos
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPRegressor
from sklearn.neural_network import MLPClassifier

# Import de métricas
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
from sklearn.metrics import accuracy_score
```

Código-fonte 1 – Imports necessários
Fonte: Elaborado pelo autor (2024)

Vamos ler o dataset mencionado anteriormente. A saída está na Figura 2:

```
# Leitura do dataset
df = pd.read_csv("insurance.csv")
df.head(5)
```

Código-fonte 2– Leitura do dataset “insurance.csv”
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Figura 2 – Saída da execução do código-fonte 2
Fonte: Elaborado pelo autor (2024)

Vamos explorar o dataset para aferir sua qualidade e descobrimos o nível de limpeza que deve ser aplicado.

```
# Quais os tipos? Dados faltantes?
df.info()
```

Código-fonte 3 – Explorando o dataset “insurance.csv”
Fonte: Elaborado pelo autor (2024)

A saída do comando abaixo está na Figura 3. Percebemos que o dataset possui 1338 linhas, sendo todas não nulas, e a maior parte dos dados já são numéricos.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Figura 3– Saída da execução do código-fonte 3
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Indo um passo além, podemos descobrir quantos valores distintos compõem as colunas categóricas. Essa informação será relevante para converter as strings em números, antes de alimentar os modelos preditivos.

```
# Quantos dados únicos
df.nunique()
```

Código-fonte 4 – Contando os dados únicos da base “insurance.csv”
Fonte: Elaborado pelo autor (2024)

A Figura 4 mostra que temos que tratar apenas dois tipos de variáveis para “sex” e “smoker”. No entanto, o tratamento de “region” exigirá a aplicação de One Hot Encoder.

```
age            47
sex             2
bmi           548
children        6
smoker          2
region          4
charges      1337
dtype: int64
```

Figura 4 – Saída da execução do código-fonte 4
Fonte: Elaborado pelo autor (2024)

Embora a base não possua valores nulos (o que facilita a limpeza, pois não precisamos realizar inputação de dados ou a exclusão das linhas problemáticas), ainda podem existir valores duplicados, o que gera problemas na divisão de treino e teste. Assim, caso existam dados duplicados, vamos excluir as cópias.

```
# Removendo duplicatas
df.drop_duplicates(inplace=True)
df.shape
```

Código-fonte 5 – Remoção de duplicatas da base “insurance.csv”
Fonte: Elaborado pelo autor (2024)

A Figura 5 mostra o tamanho da base resultante, provando que havia uma linha duplicada, a qual foi excluída (passamos de 1338 para 1337 linhas).

```
(1337, 7)
```

Figura 5 – Saída da execução do código-fonte 5
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Agora, já podemos partir para a limpeza da base, convertendo strings para números, aplicando OneHotEncoder e eliminando colunas originais que já foram tratadas. Tudo isso é feito na célula abaixo, cuja saída está na Figura 6.

```
# Convertendo dados "string" para "números"
df["genero"] = df["sex"].apply( lambda x: 0 if x == "male" else 1 )
df["fumante"] = df["smoker"].apply( lambda x: 1 if x == "yes" else 0 )

# OneHotEncoder da coluna "region"
aux = pd.get_dummies(df["region"], drop_first=True)
df_final = pd.concat([df, aux], axis=1)

# Remover colunas que já foram processadas
df_final.drop(columns=["sex", "smoker", "region"], inplace=True)
df_final
```

Código-fonte 6 – Limpeza da base “insurance.csv”
Fonte: Elaborado pelo autor (2024)

A Figura 6 mostra uma base mais bem-comportada, contendo apenas valores numéricos. No entanto, as distintas escalas das variáveis indicam a necessidade de padronização dos dados.

	age	bmi	children	charges	genero	fumante	northwest	southeast	southwest
0	19	27.900	0	16884.92400	1	1	0	0	1
1	18	33.770	1	1725.55230	0	0	0	1	0
2	28	33.000	3	4449.46200	0	0	0	1	0
3	33	22.705	0	21984.47061	0	0	1	0	0
4	32	28.880	0	3866.85520	0	0	1	0	0
...	...	...	...	...	...	...	...	...	...
1333	50	30.970	3	10600.54830	0	0	1	0	0
1334	18	31.920	0	2205.98080	1	0	0	0	0
1335	18	36.850	0	1629.83350	1	0	0	1	0
1336	21	25.800	0	2007.94500	1	0	0	0	1
1337	61	29.070	0	29141.36030	1	1	1	0	0

1337 rows × 9 columns

Figura 6 – Saída da execução do código-fonte 6
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Mas antes de padronizar os dados, é imprescindível separar as *features* do *label*, bem como os dados de TREINO e TESTE. Uma vez dividida a base nesses dois conjuntos disjuntos, aplicamos a padronização. A próxima célula não gera saída.

```
# Separação de DADOS e LABEL
X = df_final.drop(columns=["charges"])
y = df_final["charges"]

# Dividindo dados para TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Padronização dos dados
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

Código-fonte 7– Separação de dados da base “insurance.csv”
Fonte: Elaborado pelo autor (2024)

Uma vez com dados padronizados, vamos construir uma primeira versão de modelo preditivo. Como se trata de um problema de regressão, escolhemos como baseline a regressão mais simples possível, ou seja, a regressão linear. A célula abaixo não gera saída.

```
# Treinando o modelo
modelo = LinearRegression()
modelo.fit(X_train_scaled, y_train)

# Fazendo as predições
y_pred = modelo.predict(X_test_scaled)
```

Código-fonte 8– Gerando o modelo baseline (regressão linear)
Fonte: Elaborado pelo autor (2024)

Para encerrar esse bloco inicial, precisamos avaliar o modelo gerado. A célula abaixo define uma função que calcula métricas de regressão em um modelo já treinado. As métricas são reportadas tanto no TREINO quanto no TESTE. A célula abaixo não gera saída.

```
def metricas(X_tr_scaled, y_tr, y_ts, y_pr, model, id_modelo):
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
  y_pr_tr = model.predict(X_tr_scaled)

  # Erro quadrático médio
  print(f"MSE do TREINO ({id_modelo}): ", mean_squared_error(y_tr,
y_pr_tr))
  print(f"MSE do TESTE  ({id_modelo}): ", mean_squared_error(y_ts,
y_pr))

  # Erro absoluto médio
  print(f"MAE do TREINO ({id_modelo}): ", mean_absolute_error(y_tr,
y_pr_tr))
  print(f"MAE do TESTE  ({id_modelo}): ", mean_absolute_error(y_ts,
y_pr))

  # R²
  print(f"R² do TREINO ({id_modelo}): ", r2_score(y_tr, y_pr_tr))
  print(f"R² do TESTE  ({id_modelo}): ", r2_score(y_ts, y_pr))
```

Código-fonte 9 – Função de cálculo de métricas
Fonte: Elaborado pelo autor (2024)

Para aplicar a função no modelo baseline, basta chamar:

```
# Metricas para regressão linear
metricas(X_train_scaled, y_train, y_test, y_pred, modelo, "Reg
Linear")
```

Código-fonte 10 – Métricas para a regressão linear
Fonte: Elaborado pelo autor (2024)

Cuja saída está na Figura 7:

```
MSE do TREINO (Reg Linear):  35812933.23617429
MSE do TESTE  (Reg Linear):  38940169.922826335
MAE do TREINO (Reg Linear):  4165.830851177667
MAE do TESTE  (Reg Linear):  4181.815956942305
R² do TREINO (Reg Linear):  0.7362637207024127
R² do TESTE  (Reg Linear):  0.7724363518631285
```

Figura 7 – Saída da execução do código-fonte 10
Fonte: Elaborado pelo autor (2024)

As métricas da Figura 7 não estão tão ruins, indicando que a regressão linear simples já é uma boa solução inicial para o problema. Mas queremos saber se podemos ir além!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.1.3 Primeiro modelo com Rede Neural Artificial

Agora que temos um modelo baseline, verificaremos que a RNA apresentará uma solução melhor ao permitir explorar relações não lineares nos dados. Para isso, construiremos uma RNA de regressão, utilizando todos os valores *default* dos hiperparâmetros (consulte-os na documentação da classe "**MLPRegressor**"). A única alteração será no número máximo de épocas, que será limitado em 1000.

```
# Treinando uma RNA padrão e realizando predições
modelo_rn = MLPRegressor(max_iter=1000)
modelo_rn.fit(X_train_scaled, y_train)
y_pred_rn = modelo_rn.predict(X_test_scaled)

# Metricas para RNA
metricas(X_train_scaled, y_train, y_test, y_pred_rn, modelo_rn, "RN")
```

Código-fonte 11 – Gerando o modelo RNA preliminar
Fonte: Elaborado pelo autor (2024)

A saída está na Figura 8:

```
MSE do TREINO (RN):  139914070.91877007
MSE do TESTE  (RN):  171807982.1477387
MAE do TREINO (RN):  8150.72259128645
MAE do TESTE  (RN):  8836.054240466432
R² do TREINO (RN):  -0.030365657070011265
R² do TESTE  (RN):  -0.004033913412778922
```

Figura 8 – Saída da execução do código-fonte 11
Fonte: Elaborado pelo autor (2024)

2.1.4 Aprimorando o modelo de Rede Neural Artificial

O resultado do modelo RNA preliminar ficou horrível! Foi pior que o baseline! Isso indica que a arquitetura proposta (valores default dos hiperparâmetros) não captou os padrões existentes nos dados.

Portanto, devemos mudar a arquitetura proposta experimentando outras configurações de hiperparâmetros. Podemos fazer isso de diversas formas, por exemplo, copiando e colando o Código-fonte 11, mas alterando os hiperparâmetros, ou de um modo mais eficiente!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Estamos diante da necessidade de otimizar os hiperparâmetros de um modelo de machine learning. Felizmente, o sklearn já possui uma classe que facilita essa busca pela melhor combinação. Varemos isso via o código abaixo, **o qual pode demorar alguns minutos para finalizar!** Sabe dizer o motivo da demora?

```
# Otimização de hiperparâmetros para melhorar
# o resultado da predição via MLPRegressor

# Parâmetros a serem experimentados
parameters = {
    'hidden_layer_sizes': [(5,), (10,), (10, 5,), (20, 10,)],
    'activation': ['relu', 'logistic', 'tanh'],
    'solver': ['adam', 'lbfgs'],
    'alpha': [0.0001, 0.05],
    'learning_rate': ['constant', 'adaptive'],
}

# Construção e treino do grid
clf = GridSearchCV(modelo_rn, parameters, cv=5,
                   refit=True, return_train_score=True)
clf.fit(X_train_scaled, y_train)

# Melhor modelo encontrado
print("Melhores parâmetros:")
print(clf.best_params_)

# Avaliação do melhor modelo encontrado
modelo_rn_otimizado = clf.best_estimator_
y_pred_rn_otimizado = modelo_rn_otimizado.predict(X_test_scaled)
metricas(X_train_scaled, y_train, y_test,
         y_pred_rn_otimizado, modelo_rn_otimizado, "RN OTIMIZADO")
```

Código-fonte 12 – Tunando o modelo RNA preliminar
Fonte: Elaborado pelo autor (2024)

A demora na execução da última célula é que ela está treinando diversos modelos derivados das combinações de valores dos hiperparâmetro apontados. Além disso, a avaliação entre combinações distintas é feita sob o protocolo de *5-fold cross-validation*. No final das contas, temos **4** (arquiteturas de camadas ocultas) *** 3** (funções de ativação) *** 2** (otimizadores) *** 2** (reguarizações) *** 2** (protocolos de atualização da taxa de aprendizagem) *** 5** (folds) = 480 modelos sendo construídos! O que resta é esperar... A saída está na Figura 9:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
Melhores parâmetros:
{'activation': 'relu', 'alpha': 0.0001, 'hidden layer_sizes': (10, 5),
 'learning_rate': 'adaptive', 'solver': 'lbfgs'}

MSE do TREINO (RN OTIMIZADO):  19470331.2606868
MSE do TESTE  (RN OTIMIZADO):  29480945.388350822
MAE do TREINO (RN OTIMIZADO):  2568.19811607923
MAE do TESTE  (RN OTIMIZADO):  3149.3141631383705
R² do TREINO (RN OTIMIZADO):  0.8566151314799819
R² do TESTE  (RN OTIMIZADO):  0.827715402978651
```

Figura 9 – Saída da execução do código-fonte 12
Fonte: Elaborado pelo autor (2024)

2.1.5 Considerações finais

Nessa análise de modelagem preditiva, foram testados três diferentes modelos para resolver um problema específico. O primeiro modelo, uma simples re*gressão linear*, foi a primeira abordagem adotada. Embora seja um método básico, é uma escolha comum para problemas iniciais de predição. No entanto, os resultados mostraram que a regressão linear não capturou adequadamente a complexidade dos dados, resultando em um desempenho inferior.

Diante disso, foi implementada uma *rede neural artificial (RNA) padrão* como segunda tentativa. Esperava-se que a RNA fosse capaz de aprender padrões mais complexos nos dados, mas, surpreendentemente, o desempenho do modelo ficou aquém das expectativas. Foi então que a aplicação de *otimização de hiperparâmetros via GridSearch*, no terceiro modelo, revelou-se crucial. Esse processo permitiu explorar uma variedade de configurações de hiperparâmetros da RNA e identificar a combinação mais eficaz para o problema em questão.

Esse resultado ressalta a importância de testar diferentes arquiteturas e ajustes em redes neurais artificiais, destacando a necessidade de abordagens experimentais e iterativas na modelagem preditiva.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Aplicação em Marketing

As análises preditivas têm uma variedade de aplicações no marketing, ajudando as empresas a entenderem o comportamento do consumidor, otimizar estratégias de vendas e melhorar o retorno sobre o investimento em publicidade.

O potencial das aplicações é vasto, incluindo, mas não se limitando, a previsão de demanda, segmentação de mercado, recomendação de produtos e otimização de preços.

As aplicações listadas podem ser tratadas como problemas supervisionados, não supervisionados e até mesmo análises estatísticas clássicas. Tudo depende do objetivo principal da demanda.

2.2.1 Definição do problema e dos dados

Nosso case tem como objetivo representar o impacto dos investimentos em publicidade nos resultados de vendas de um produto ou serviço. Esse problema é relevante no marketing porque as empresas precisam entender como a alocação de recursos em diferentes canais de publicidade afeta suas vendas, para otimizar suas estratégias de marketing e maximizar o retorno sobre o investimento.

DICA: Para essa parte do código, utilizaremos o arquivo “advertising.csv”!

Para essa modelagem, utilizaremos outra base de dados clássica. As variáveis que a compõem são:

- **TV**: valor gasto em publicidade na televisão;
- **Radio**: valor gasto em publicidade no rádio;
- **Newspaper**: valor gasto em publicidade em jornais;
- **Sales**: número de vendas do produto ou serviço.

Essas variáveis representam os diferentes canais de publicidade e as vendas associadas àquela alocação, permitindo que os analistas e profissionais de marketing avaliem o impacto de cada canal nas vendas totais.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2.2 Primeiros passos

Referenciando o ciclo de vida das soluções de ciência de dados (Figura 1), já passamos pela etapa de entendimento do problema e obtenção dos dados. Agora, já aplicando programação, faremos a *leitura*, *exploração* e *preparação dos dados*, bem como criaremos o *modelo preditivo baseline*. Recomendamos que você programe junto conosco!

Não há imports a serem feitos, pois já os realizamos no início do primeiro case. Vamos ler o dataset e observar a saída na Figura 10:

```
# Lendo o dataset
df = pd.read_csv("advertising.csv")
df.head()
```

Código-fonte 13 – Leitura do dataset “advertising.csv”
Fonte: Elaborado pelo autor (2024)

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

Figura 10 – Saída da execução do código-fonte 13
Fonte: Elaborado pelo autor (2024)

Vamos explorar o dataset para aferir sua qualidade e descobrimos o nível de limpeza que deve ser aplicado.

```
# Quais os tipos? Dados faltantes?
df.info()
```

Código-fonte 14 – Explorando o dataset “advertising.csv”
Fonte: Elaborado pelo autor (2024)

A saída do comando abaixo está na Figura 11. Percebemos que o dataset possui 200 linhas, sendo todas não nulas, e todos os dados já são numéricos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

Figura 11 – Saída da execução do código-fonte 14
Fonte: Elaborado pelo autor (2024)

Apesar da base ser pequena, para garantir a correta separação de dados em treino e teste, vamos eliminar qualquer dado duplicado, notando, na Figura 12, que a base continua do mesmo tamanho, portanto, não há dados duplicados.

```
# Removendo duplicatas
df.drop_duplicates(inplace=True)
df.shape
```

Código-fonte 15 – Removendo dados duplicados do dataset “advertising.csv”
Fonte: Elaborado pelo autor (2024)

```
(200, 4)
```

Figura 12 – Saída da execução do código-fonte 15
Fonte: Elaborado pelo autor (2024)

Como os dados dessa base estão bem-comportados, já podemos partir para a separação de *features* e *label*, bem como dividir os exemplos entre TREINO e TESTE. Uma vez dividida a base nesses dois conjuntos disjuntos, aplicamos a padronização. A próxima célula não gera saída.

```
# Separação de DADOS e LABEL
X = df.drop(columns=["Sales"])
y = df["Sales"]

# Dividindo dados para TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
)

# Padronização dos dados
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

Código-fonte 16 – Separação de dados da base “advertising.csv”
Fonte: Elaborado pelo autor (2024)

Uma vez com dados padronizados, vamos construir uma primeira versão de modelo preditivo. Novamente, empregaremos a regressão linear, já avaliando o modelo resultante via a função “*metricas()*” que construímos anteriormente.

```
# Treinando o modelo
modelo = LinearRegression()
modelo.fit(X_train_scaled, y_train)

# Fazendo as predições
y_pred = modelo.predict(X_test_scaled)

# Metricas para regressao linear
metricas(X_train_scaled, y_train, y_test, y_pred, modelo, "Reg
Linear")
```

Código-fonte 17 – Gerando o modelo baseline (regressão linear)
Fonte: Elaborado pelo autor (2024)

```
MSE do TREINO (Reg Linear):  2.8392326607544245
MSE do TESTE  (Reg Linear):  2.54162403622915
MAE do TREINO (Reg Linear):  1.2836301223784408
MAE do TESTE  (Reg Linear):  1.1594875061090584
R² do TREINO (Reg Linear):  0.8919647339302077
R² do TESTE  (Reg Linear):  0.9091484341849798
```

Figura 13 – Saída da execução do código-fonte 17
Fonte: Elaborado pelo autor (2024)

As métricas da Figura 13 estão boas! Isso indica que os dados possuem uma relação próxima à linear. Mas queremos saber se podemos ir além!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2.3 Primeiro modelo com Rede Neural Artificial

Agora que temos um modelo baseline, verificaremos se a RNA apresentará uma solução melhor ao permitir explorar relações não lineares nos dados. Para isso, construiremos uma RNA de regressão, utilizando todos os valores default dos hiperparâmetros (consulte-os na documentação da classe “MLPRegressor”). A única alteração será no número máximo de épocas, que será limitado em 1000.

```
# Treinando uma RNA padrão e realizando predições
modelo_rn = MLPRegressor(max_iter=1000, random_state=97)
modelo_rn.fit(X_train_scaled, y_train)
y_pred_rn = modelo_rn.predict(X_test_scaled)

# Metricas para RNA otimizada
metricas(X_train_scaled, y_train, y_test, y_pred_rn, modelo_rn, "RN")
```

Código-fonte 18 – Gerando o modelo RNA preliminar
Fonte: Elaborado pelo autor (2024)

```
MSE do TREINO (RN):  3.3393236244781193
MSE do TESTE  (RN):  5.636869802488712
MAE do TREINO (RN):  1.381398811609204
MAE do TESTE  (RN):  1.6621776019409256
R² do TREINO (RN):  0.8729358388798696
R² do TESTE  (RN):  0.7985073950546592
```

Figura 14 – Saída da execução do código-fonte 18
Fonte: Elaborado pelo autor (2024)

2.2.4 Aprimorando o modelo de Rede Neural Artificial

Novamente, o primeiro modelo baseado em RNA não saiu melhor que o baseline, portanto, temos que investigar configurações de hiperparâmetros que fazem mais sentido aos dados apresentados.

Por exemplo, podemos tunar a rede aprimorando sua arquitetura, alterando o otimizador e a política de adaptação da taxa de aprendizagem.

```
# Treinando uma RNA padrão e realizando predições
modelo_otimizado = MLPRegressor(hidden_layer_sizes=(10, 5,),
                                max_iter=1000,
                                solver="lbfgs",
                                learning_rate="adaptive",
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                            random_state=97)
modelo_otimizado.fit(X_train_scaled, y_train)
y_pred_rn = modelo_otimizado.predict(X_test_scaled)

# Metricas para RNA otimizada
metricas(X_train_scaled, y_train, y_test, y_pred_rn, modelo_otimizado,
"RN OTIMIZADO")
```

Código-fonte 19 – Gerando o modelo RNA secundário – Seed 97
Fonte: Elaborado pelo autor (2024)

A Figura 15 mostra que o resultado melhorou, reforçando o conceito que os hiperparâmetros das redes neurais multicamadas devem ser investigados para melhor configuração do modelo.

```
MSE do TREINO (RN OTIMIZADO):  0.4379354351237677
MSE do TESTE  (RN OTIMIZADO):  1.9127624091307802
MAE do TREINO (RN OTIMIZADO):  0.4511983909532657
MAE do TESTE  (RN OTIMIZADO):  0.982569649097093
R² do TREINO (RN OTIMIZADO):  0.9833361767392409
R² do TESTE  (RN OTIMIZADO):  0.9316273935780587
```

Figura 15 – Saída da execução do código-fonte 19
Fonte: Elaborado pelo autor (2024)

Por outro lado, o que realmente fez diferença no código acima foi a semente aleatória utilizada "`random_state=97`". Se nós rodássemos o mesmo código, porém alterando o valor da semente, por exemplo, para "`random_state=93`", teríamos outro resultado:

```
# Treinando uma RNA padrão e realizando predições
modelo_otimizado = MLPRegressor(hidden_layer_sizes=(10, 5,),
                            max_iter=1000,
                            solver="lbfgs",
                            learning_rate="adaptive",
                            random_state=93)
modelo_otimizado.fit(X_train_scaled, y_train)
y_pred_rn = modelo_otimizado.predict(X_test_scaled)

# Metricas para RNA otimizada
metricas(X_train_scaled, y_train, y_test, y_pred_rn, modelo_otimizado,
"RN OTIMIZADO")
```

Código-fonte 20 – Gerando o modelo RNA secundário – Seed 93
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
MSE do TREINO (RN OTIMIZADO):  0.7223982401992917
MSE do TESTE  (RN OTIMIZADO):  4.93884563479431
MAE do TREINO (RN OTIMIZADO):  0.5615115916845373
MAE do TESTE  (RN OTIMIZADO):  1.6007727270138643
R² do TREINO (RN OTIMIZADO):  0.9725121202052027
R² do TESTE  (RN OTIMIZADO):  0.8234586025140637
```

Figura 16 – Saída da execução do código-fonte 20
Fonte: Elaborado pelo autor (2024)

Agora executando novamente com “`random_state=3`”:

```
# Treinando uma RNA padrão e realizando predições
modelo_otimizado = MLPRegressor(hidden_layer_sizes=(10, 5,),
                        max_iter=1000,
                        solver="lbfgs",
                        learning_rate="adaptive",
                        random_state=3)
modelo_otimizado.fit(X_train_scaled, y_train)
y_pred_rn = modelo_otimizado.predict(X_test_scaled)

# Metricas para RNA otimizada
metricas(X_train_scaled, y_train, y_test, y_pred_rn, modelo_otimizado,
"RN OTIMIZADO")
```

Código-fonte 21 – Gerando o modelo RNA secundário – Seed 3
Fonte: Elaborado pelo autor (2024)

```
MSE do TREINO (RN OTIMIZADO):  0.7093816540612269
MSE do TESTE  (RN OTIMIZADO):  4.145685663426665
MAE do TREINO (RN OTIMIZADO):  0.6242040023304216
MAE do TESTE  (RN OTIMIZADO):  1.5046654466233291
R² do TREINO (RN OTIMIZADO):  0.9730074126009912
R² do TESTE  (RN OTIMIZADO):  0.8518104847410896
```

Figura 17 – Saída da execução do código-fonte 21
Fonte: Elaborado pelo autor (2024)

As Figura 16 e Figura 17 mostram uma piora do modelo pelo simples fato de alterarmos a semente de inicialização aleatória utilizada no treinamento. Essa semente influencia, entre outras coisas, a inicialização dos pesos da rede. Dependendo da inicialização, pode ser mais “fácil” ou mais “difícil” treinar a rede, atingindo ótimos locais distintos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2.5 Considerações finais

A escolha do seed aleatório desempenha um papel crucial na construção de redes neurais multicamadas, especialmente durante a inicialização dos pesos. O seed determina a sequência de números pseudoaleatórios gerados durante a inicialização dos pesos da rede, o que afeta diretamente o estado inicial da rede e, por conseguinte, seu desempenho e comportamento durante o treinamento. Uma escolha inadequada do seed pode levar a resultados inconsistentes e não reproduzíveis, dificultando a comparação entre diferentes configurações de rede e tornando o processo de treinamento menos confiável.

Além disso, a escolha do seed aleatório pode influenciar os desafios enfrentados durante o treinamento de redes neurais, como o problema do gradiente desaparecendo ou explodindo. Inicializações inadequadas dos pesos podem levar a gradientes muito pequenos ou muito grandes, o que dificulta a convergência do algoritmo de otimização durante o treinamento. Portanto, ao escolher o seed aleatório, os praticantes de aprendizado de máquina devem considerar cuidadosamente a inicialização dos pesos da rede e garantir que ela seja feita de maneira apropriada para evitar esses desafios e facilitar o processo de treinamento da rede.

Ao escolher o seed para o treinamento de RNAs, considere as seguintes **dicas**:

- **Reprodutibilidade**: escolha um seed que permita a reprodução dos resultados. Isso é essencial para garantir consistência nos experimentos e facilitar a comparação entre diferentes configurações de RNA;
- **Experimentação**: teste diferentes valores de seed para avaliar seu impacto no desempenho da RNA. Varie o seed em pequenos incrementos para observar como os resultados mudam e identificar o melhor valor para o seu problema específico;
- **Aleatoriedade**: o seed deve ser escolhido de forma aleatória para evitar padrões indesejados nos dados de treinamento. Utilize métodos de geração de números pseudoaleatórios confiáveis para garantir a aleatoriedade;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Documentação**: registre o seed escolhido juntamente com os resultados do experimento. Isso ajudará a garantir a transparência e a replicabilidade do estudo;
- **Consistência**: mantenha o mesmo seed ao longo de diferentes execuções do treinamento para garantir a consistência nos resultados. Isso é especialmente importante ao comparar diferentes abordagens ou algoritmos de RNA.

3 PROBLEMAS DE CLASSIFICAÇÃO

3.1 Aplicação em Recursos Humanos

Outra área cuja utilização de dados tem aumentado nos últimos anos é no departamento de Recursos Humanos (RH). A tomada de decisão inteligente é fundamental para promover uma gestão mais estratégica e eficiente das pessoas nas organizações.

Por meio da análise de dados, conhecida como **People Analytics**, as empresas podem extrair insights valiosos sobre seus colaboradores e processos internos, permitindo uma abordagem mais objetiva e baseada em evidências para questões relacionadas ao capital humano.

People analytics envolve a coleta, análise e interpretação de dados relacionados aos colaboradores de uma organização, com o objetivo de informar e otimizar processos de RH. Alguns exemplos de aplicações de People Analytics incluem:

- **Recrutamento e seleção**: utilização de dados para identificar candidatos com o perfil mais adequado para determinadas vagas, considerando características como habilidades, experiência e compatibilidade cultural;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Engajamento e satisfação**: monitoramento do engajamento dos colaboradores por meio de pesquisas de clima organizacional e análise de indicadores como taxa de turnover e feedbacks internos, visando identificar áreas de melhoria e promover um ambiente de trabalho mais satisfatório;
- **Desenvolvimento e capacitação**: identificação das necessidades de treinamento e desenvolvimento dos colaboradores com base em avaliações de desempenho, feedbacks e análise de lacunas de habilidades, para oferecer programas de capacitação mais direcionados e eficazes;
- **Planejamento de sucessão**: identificação e desenvolvimento de talentos internos com potencial para ocupar posições de liderança no futuro, por meio da análise de desempenho, competências e histórico de carreira.

Enfim, as aplicabilidades são vastas! Vamos conhecer outra neste terceiro e último case do capítulo.

3.1.1 Definição do problema e dos dados

O problema em questão é de classificação binária, onde o objetivo é **prever se um colaborador irá deixar a empresa ou não**. Essa previsão é útil para o departamento de recursos humanos, pois permite identificar antecipadamente os funcionários com maior probabilidade de sair, possibilitando a implementação de medidas preventivas para retenção de talentos.

DICA: Para essa parte do código, utilizaremos o arquivo “turnover.csv”!

As variáveis que compõem a base e suas respectivas descrições são:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **satisfaction_level**: nível de satisfação do colaborador com o trabalho, geralmente medido por meio de pesquisas de satisfação ou avaliações internas;
- **last_evaluation**: última avaliação de desempenho do colaborador, que pode ser realizada periodicamente pela empresa para acompanhar o progresso e a performance;
- **number_project**: número de projetos em que o colaborador está atualmente envolvido, fornecendo uma medida da carga de trabalho e da diversidade de responsabilidades;
- **average_montly_hours**: média de horas trabalhadas por mês pelo colaborador, um indicador do seu nível de envolvimento e dedicação ao trabalho;
- **time_spend_company**: tempo de permanência do colaborador na empresa, em anos, que pode influenciar sua propensão a deixar a organização;
- **Work_accident**: indicador binário que representa se o colaborador sofreu algum acidente de trabalho durante seu período na empresa;
- **left**: variável de destino binária que indica se o colaborador deixou a empresa (1) ou permaneceu (0);
- **promotion_last_5years**: indica se o colaborador recebeu alguma promoção nos últimos cinco anos, o que pode influenciar sua satisfação e lealdade;
- **department**: departamento em que o colaborador trabalha, fornecendo informações sobre a área de atuação e a equipe em que está inserido;
- **salary**: nível salarial do colaborador, que pode ser classificado em baixo, médio ou alto, oferecendo uma indicação do seu status e remuneração na empresa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3.1.2 Primeiros passos

Novamente, podemos partir para o carregamento de dados, exploração e limpeza. Aproveitaremos para fazer o modelo inicial (baseline). Não há necessidade de novos imports, pois todos foram importados no início do capítulo.

```
# Carregando o dataset
data = pd.read_csv('turnover.csv')
data.head()
```

Código-fonte 22 – Carregando do dataset “turnover.csv”
Fonte: Elaborado pelo autor (2024)

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left
0	0.38	0.53	2	157	3	0	1
1	0.80	0.86	5	262	6	0	1
2	0.11	0.88	7	272	4	0	1
3	0.72	0.87	5	223	5	0	1
4	0.37	0.52	2	159	3	0	1

Figura 18 – Saída parcial (colunas suprimidas) da execução do código-fonte 22
Fonte: Elaborado pelo autor (2024)

Nesse tipo de problema, é interessante notar a distribuição dos exemplos entre as classes, pois esperamos que a base seja desbalanceada;

```
# Plotando a distribuição das classes
plt.figure()
sns.countplot(x='left', data=data)

plt.xlabel('Deixou a companhia?', fontsize=13)
plt.xticks(ticks=[0, 1], labels=["Não (0)", "Sim (1)"])
plt.ylabel('Quantidade', fontsize=13)
plt.title('Contagem dos exemplos de cada classe', fontsize=15)
plt.show()
```

Código-fonte 23 – Plotando a distribuição das classes no dataset
Fonte: Elaborado pelo autor (2024)

A Figura 19 mostra o que esperávamos: mais pessoas ficaram na empresa do que saíram. Quando há um desbalanceamento grande entre as classes, os resultados podem ser penalizados. Para isso, existem técnicas para tratamento da base. Esse tratamento, neste capítulo, está fora do escopo, e podemos seguir com a análise.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

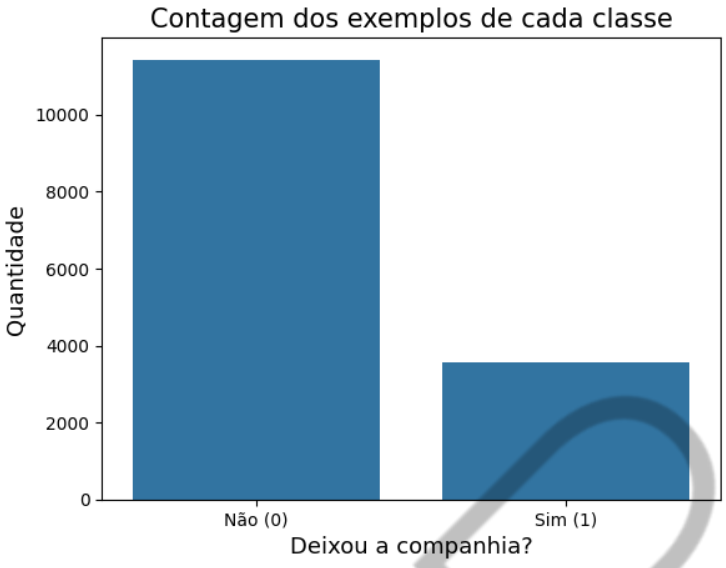

Figura 19 – Saída da execução do código-fonte 23
Fonte: Elaborado pelo autor (2024)

Limpeza do dataset, excluindo linhas duplicadas, e convertendo os dados textuais em números. A célula abaixo não produz saída.

```
# Removendo duplicatas
data.drop_duplicates(inplace=True)

# Criando o LabelEncoder
le = LabelEncoder()

# Convertendo strings em números
data['salary'] = le.fit_transform(data['salary'])
data['department'] = le.fit_transform(data['department'])
```

Código-fonte 24 – Limpando os dados
Fonte: Elaborado pelo autor (2024)

Agora, podemos separar o label (left) das features (todas as demais colunas), bem como dividir a base entre TREINO e TESTE. Aproveitaremos para já padronizar os dados. A célula abaixo não produz saída.

```
# Separando features do label
X = data.drop(columns=["left"])
y = data["left"]

# Dividindo os dados em TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    test_size=0.3,
    random_state=42)

# Padronização dos dados
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

Código-fonte 25 – Preparando os dados para aplicação de Machine Learning
Fonte: Elaborado pelo autor (2024)

E na sequência, **criaremos nosso modelo baseline**. Também usaremos uma estratégia de separação linear, aplicando a Regressão Logística:

```
# Modelo baseline: regressão logística
logreg = LogisticRegression()
logreg.fit(X_train_scaled, y_train)
y_pred_logreg = logreg.predict(X_test_scaled)

# Calcula a acurácia da predição
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
print("ACC (regressão logística):", accuracy_logreg)
```

Código-fonte 26 – Criando o modelo baseline
Fonte: Elaborado pelo autor (2024)

A saída está na Figura 20:

```
ACC (regressão logística): 0.8315730961645359
```

Figura 20 – Saída da execução do código-fonte 26
Fonte: Elaborado pelo autor (2024)

3.1.3 Primeiro modelo com Rede Neural Artificial

Vamos aprimorar nosso preditor de turnover aplicando uma RNA. A arquitetura dessa rede é composta por 2 camadas ocultas, uma com 6 neurônios e a outra, com 5. A taxa de aprendizado inicial é 0.01. Os demais hiperparâmetros seguirão seus valores *default*:

```
# Modelo baseado em RNA
clf = MLPClassifier(hidden_layer_sizes=(6, 5),
                    random_state=5,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                     learning_rate_init=0.01)
clf.fit(X_train_scaled, y_train)
ypred=clf.predict(X_test_scaled)

# Calcula a acurácia da predição
accuracy_clf = accuracy_score(y_test, ypred)
print("ACC (rede neural):", accuracy_clf)
```

Código-fonte 27 – Criando o modelo baseline
Fonte: Elaborado pelo autor (2024)

A saída está na Figura 21:

```
ACC (rede neural): 0.9660922734852696
```

Figura 21– Saída da execução do código-fonte 27
Fonte: Elaborado pelo autor (2024)

Notamos que houve uma melhoria significativa, e que poderia ser ainda melhor se realizássemos a otimização de hiperparâmetros! Deixaremos essa tarefa para você!

3.1.4 Aprimorando o modelo de Rede Neural Artificial

Sabemos que ao trocar o algoritmo preditivo por outro mais complexo, o resultado foi melhor. Agora, você já sabe como aprimorar ainda mais o modelo baseado em RNA. Então, esse trecho de código se dedica a explicar o que levou à melhoria da predição. Faremos uma explicação visual!

O primeiro passo é plotar os dados do dataset num gráfico, diferenciando por cores os exemplos de turnover (left = 1) dos que permaneceram na empresa (left = 0). No entanto, nosso dataset possui 10 variáveis! Como plotar um gráfico com tantas informações?

Para resolver essa questão, ou podemos plotar os gráficos selecionando 2 a 2 variáveis ou podemos tentar sintetizar as 10 variáveis em apenas 2 dimensões. Faremos a segunda alternativa aplicando *Principal Component Analysis* (PCA).

```
# Aplicar PCA com 2 componentes principais
pca = PCA(n_components=2)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
X_pca = pca.fit_transform(X_train_scaled)

# Criando um DataFrame com as componentes princiais do PCA e a coluna
"left"
X_pca = pd.DataFrame(data=X_pca, columns=['pc1', 'pc2'])
X_pca['left'] = y_train

# Plotando os dados em 2 dimensões, diferenciando pelo "left"
sns.scatterplot(x='pc1', y='pc2', hue='left', alpha=0.5,
data=X_pca.sample(2500))
plt.xlabel('PC1', fontsize=13)
plt.ylabel('PC2', fontsize=13)
plt.title('Dispersão dos pontos nas 2 dimensões do PCA', fontsize=15)
plt.show()
```

Código-fonte 28 – Explicação do comportamento não linear do dataset
Fonte: Elaborado pelo autor (2024)

A Figura 22 mostra a plotagem de 2500 exemplos aleatórios do treino. O parâmetro “” indica a transparência das bolinhas coloridas, caso uma bolinha seja plotada em cima da outra. O gráfico indica que o dataset não possui uma separação óbvia dos labels, ou seja, não há um espaço claramente determinado por “pessoas que saíram da empresa”. Portanto, há uma forte confusão (mistura) de exemplos nas 2 classes, o que torna a predição mais desafiadora.

Se houvesse uma separação clara, a regressão logística proporia um “separador linear” com bom acerto, mas não é o caso. Assim, precisamos utilizar um modelo mais complexo que explore padrões sutis entre os dados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

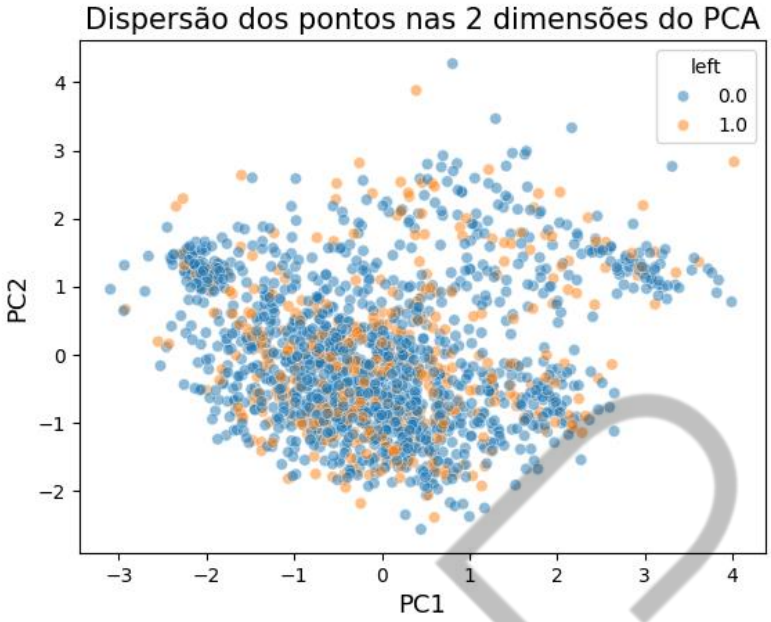

Figura 22 – Saída da execução do código-fonte 28
Fonte: Elaborado pelo autor (2024)

3.1.5 Considerações finais

Concluindo esse case, é importante ressaltar que se trata de um problema complexo e não linear. Ao explorar e visualizar os dados durante o processo de modelagem, é essencial observar a distribuição das classes de saída e verificar se há ou não uma separação completa entre elas.

Em outras palavras, é importante verificar se existem regiões no espaço de características onde as instâncias positivas e negativas se sobrepõem, pois isso pode indicar a necessidade de modelos mais complexos e flexíveis para capturar essas nuances e evitar o subajuste dos dados.

Portanto, o uso de técnicas de visualização de dados, como gráficos de dispersão ou mapas de calor, bem como o emprego do PCA, pode ser valioso para auxiliar na escolha da complexidade do algoritmo preditivo e garantir que o modelo seja capaz de generalizar efetivamente para novos dados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4 CONCLUSÕES

Neste capítulo, conhecemos mais três aplicações de RNAs, sendo o primeiro contato com redes multicamadas para regressão. Também aproveitamos para reforçar conceitos e introduzir outras práticas que auxiliam na construção de bons preditores baseados em redes neurais artificiais.

IMPORTANTE: as três principais lições deste capítulo são:

- Construir boas arquiteturas de RNAs não é uma tarefa fácil e exige experimentação!

- Atenção especial à reprodutibilidade dos resultados e fatores aleatórios do treinamento de RNAs!

- Visualizar a dispersão de dados em dimensões menores pode auxiliar a detectar o grau de complexidade dos modelos exigida pelos dados!

Treinar redes neurais multicamadas de forma efetiva pode ser desafiador, mas algumas dicas podem ajudar a alcançar melhores resultados. Aqui estão elas:

- **Inicialização dos pesos**: escolha cuidadosamente as sementes aleatórias ou o método de inicialização dos pesos da rede, evitando problemas de explosão ou desaparecimento de gradiente;
- **Otimização de hiperparâmetros**: experimente diferentes configurações de hiperparâmetros para encontrar a combinação ideal que maximize o desempenho da rede. Isso inclui ajustar a taxa de aprendizado, o tamanho do lote, o número de épocas, o algoritmo de otimização, entre outros;
- **Validação cruzada**: utilize a validação cruzada para avaliar o desempenho do modelo de forma mais robusta e evitar o sobreajuste aos dados de treinamento. Isso ajudará a garantir que o modelo generalize bem para novos dados;
- **Regularização**: considere o uso de técnicas de regularização, como L1, L2 ou dropout, para evitar o sobreajuste e melhorar a capacidade de generalização do modelo;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Monitoramento do desempenho**: acompanhe o desempenho da rede ao longo do treinamento, monitorando métricas como função de perda e acurácia. Isso pode ajudar a identificar problemas precocemente e ajustar a estratégia de treinamento conforme necessário;
- **Criação de baselines**: estabeleça baselines de desempenho com modelos simples antes de prosseguir com arquiteturas mais complexas. Isso fornecerá uma referência para avaliar o progresso e a eficácia das melhorias implementadas;
- **Visualização dos resultados**: use visualizações, como curvas de aprendizado e matrizes de confusão, para entender melhor o comportamento do modelo e identificar áreas de melhoria;
- **Experimentação iterativa**: adote uma abordagem iterativa para o treinamento, realizando experimentos incrementais e ajustando a estratégia com base nos resultados obtidos.

Ao seguir essas dicas e manter uma abordagem sistemática e experimental, é possível treinar redes neurais multicamadas de forma mais efetiva e alcançar melhores resultados em tarefas de aprendizado de máquina.

5 HANDS ON: AVANÇANDO OS MODELOS CONSTRUÍDOS

5.1 Seu desafio

Agora é a sua vez de criar novos modelos de regressão e classificação com Redes Neurais Multicamadas. Utilizando como base os códigos e datasets vistos no material, sua tarefa será:

1. Construir novos modelos baseados em RNAs para os problemas apresentados, alterando a seleção e normalização de features;
2. Otimizar outros parâmetros da base de marketing;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3. Pesquisar as semelhanças entre o problema de turnover e churn.

5.2 Proposta de solução

Você já está apto(a) para desenvolver os desafios [1] e [2] de forma autônoma. As possibilidades de solução são diversas. Sua tarefa será comparar seu resultado com o apresentado no material, tendo a visão crítica de compreender porque eles ficaram melhores ou piores.

Quanto ao item [3], temos:

O termo "churn", em geral, refere-se à taxa de rotatividade ou abandono de clientes em uma empresa, especialmente em serviços como telecomunicações, assinaturas de software, streaming de mídia e outros setores onde os clientes têm uma relação contínua com a empresa. No contexto empresarial, o churn representa o número de clientes que cancelam seus contratos ou deixam de utilizar os serviços oferecidos pela empresa durante um determinado período.

Por outro lado, o problema de abandono do emprego, ou "turnover" no contexto de recursos humanos, refere-se à taxa de rotatividade de funcionários em uma organização, ou seja, o número de colaboradores que deixam seus empregos e precisam ser substituídos durante um período específico. Assim como o churn, o turnover é uma métrica importante para as empresas, pois pode impactar diretamente a produtividade, a cultura organizacional e os custos de recrutamento e treinamento.

Embora o churn e o turnover se refiram a contextos diferentes (clientes versus funcionários), ambos envolvem a perda de recursos importantes para a empresa e podem ter consequências significativas para o desempenho e o sucesso do negócio. Em ambos os casos, a análise preditiva pode ser empregada para identificar os fatores que contribuem para o churn ou turnover, permitindo que a empresa tome medidas proativas para mitigar essas perdas e manter a satisfação e a fidelidade dos clientes e funcionários.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

HEATON, Jeff. **Introduction to neural networks with Java**. Heaton Research, Inc., 2008.

SILVA, Ivan Nunes da; SPATTI, Danilo Hernane; FLAUZINO, Rogério Andrade. **Redes neurais artificiais para engenharia e ciências aplicadas**. 2010.

VON ZUBEN, Fernando J.; DE CASTRO, Leandro N. **Redes neurais**. Notas de aula, FEEC, Universidade Estadual de Campinas, 2003.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

RNA	Rede Neural Artificial
RH	Recursos Humanos

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com